**Bitácora de Prompts e Interacción con Inteligencia Artificial**
**Proyecto:** Sistema de Gestión de Biblioteca (UNICAUCA)
**Objetivo de uso de IA:** Agilizar la escritura de la capa de interacción en consola (UI), validaciones de flujo interactivo y vinculación de controladores con la clase principal Sistema.
**Caso 1: Estructuración del Menú Consola e Interacción Principal**

- **Prompt enviado a la IA:**

*"Tengo una clase Sistema en C++ para administrar bibliotecas, libros, estantes, estudiantes y préstamos. Necesito que me ayudes a crear la función de menú interactivo mostrarMenu y la estructura base en el main() con un bucle do-while y switch-case que soporte hasta 14 opciones diferentes y la opción 0 para salir. Incluye también una función auxiliar para limpiar el buffer de lectura de cin antes de usar getline."*

- **Respuesta de la IA:**

*"Aquí tienes la implementación de la función mostrarMenu con el listado de opciones, la función limpiarBuffer usando getline y la integración del bucle de menú principal en C++."*

- **Resultado del Código Generado (main_4.cpp):**

C++
```
// Limpia los saltos de linea pendientes en el buffer cin
void limpiarBuffer() {
   std::string temp;
   std::getline(std::cin, temp);
}

// Despliega el menu visual separando la opcion 5 en dos casos (del 1 al 14)
void mostrarMenu(const std::string& nombreInstitucion) {
   std::cout << "\n===== BIENVENIDO AL SISTEMA " << nombreInstitucion << " =====\n";
   std::cout << "MENU DE OPCIONES\n";
   std::cout << "1: Registrar un Autor\n";
   std::cout << "2: Registrar un Libro\n";
   std::cout << "3: Registrar una Estanteria nueva\n";
   std::cout << "4: Registrar un ejemplar de libro\n";
   std::cout << "5: Registrar una Biblioteca\n";
   std::cout << "6: Asignar estanteria a una biblioteca\n";
   std::cout << "7: Consultar un libro\n";
   std::cout << "8: Listado de inventario\n";
   std::cout << "9: Conteo de inventario\n";
   std::cout << "10: Actualizar el inventario\n";
   std::cout << "11: Registrar un Estudiante\n";
   std::cout << "12: Registrar prestamo de libro\n";
   std::cout << "13: Listar prestamos por Biblioteca\n";
   std::cout << "14: Listar libros prestados por Estudiante\n";
   std::cout << "0: Finalizacion de operacion del sistema\n";
   std::cout << "Seleccione una opcion: ";
}
```

**Caso 2: Flujo Interactivo Pasos a Paso para Registrar un Préstamo**

- **Prompt enviado a la IA:**

*"Necesito una función interactiva en C++ llamada ejecutarRegistroPrestamo(Sistema& sistema). Debe realizar un flujo guiado en varios pasos:

1. Mostrar y seleccionar un estudiante de la lista por su número de índice.
2. Mostrar y seleccionar una biblioteca.
3. Mostrar y seleccionar un estante pertenecientes a esa biblioteca.
4. Mostrar y seleccionar un libro disponible en ese estante.

Si en algún momento el usuario presiona 0 o elige una opción inválida, la función debe cancelar la operación e informar al usuario."*

- **Respuesta de la IA:**

*"Se crea la función ejecutarRegistroPrestamo encargada de validar la existencia de datos y encadenar secuencialmente la selección de cada entidad mediante punteros obtendidos del objeto Sistema."*

- **Resultado del Código Generado (main_4.cpp):**

C++

```
// Ejecuta la captura interactiva para registrar un prestamo
void ejecutarRegistroPrestamo(Sistema& sistema) {
   std::cout << "\n--- Registrar Prestamo de Libro ---\n";

   if (sistema.getEstudiantes().empty()) {
      std::cout << "-> No hay estudiantes registrados. Registre un estudiante primero (Opcion
11).\n";
      return;
   }

   if (sistema.getBibliotecas().empty()) {
      std::cout << "-> No hay bibliotecas registradas en el sistema.\n";
      return;
   }

   // 1. Seleccionar estudiante
   sistema.mostrarEstudiantes();
   int indiceEstudiante = 0;
   std::cout << "Seleccione el numero del estudiante (o 0 para salir): ";
   std::cin >> indiceEstudiante;

   if (indiceEstudiante == 0 || indiceEstudiante < 1 || indiceEstudiante >
(int)sistema.getEstudiantes().size()) {
      std::cout << "-> Operacion cancelada o seleccion invalida.\n";
      return;
   }

   Estudiante* estudianteSeleccionado = sistema.getEstudiantes()[indiceEstudiante - 1];
```

```
   // 2. Seleccionar biblioteca
   sistema.mostrarBibliotecas();
   int indiceBiblioteca = 0;
   std::cout << "Seleccione el numero de la biblioteca (o 0 para salir): ";
   std::cin >> indiceBiblioteca;

   if (indiceBiblioteca == 0 || indiceBiblioteca < 1 || indiceBiblioteca >
(int)sistema.getBibliotecas().size()) {
      std::cout << "-> Operacion cancelada o seleccion invalida.\n";
      return;
   }

   Biblioteca* bibliotecaSeleccionada = sistema.getBibliotecas()[indiceBiblioteca - 1];

   // 3 y 4. Seleccionar Estanteria y Libro (mismo patron de seleccion)...
   // Finaliza invocando la logica del dominio:
   // sistema.registrarPrestamo(estudianteSeleccionado, bibliotecaSeleccionada,
estanteSeleccionado, libroSeleccionado);
}
```

**Caso 3: Controladores para Consultas de Préstamos (Por Biblioteca y Estudiante)**

- **Prompt enviado a la IA:**

*"Genera dos funciones auxiliares para la interfaz de consola: ejecutarListarPrestamosPorBiblioteca y ejecutarListarPrestamosPorEstudiante. Cada una debe verificar que existan registros creados en Sistema, mostrar el listado numerado para que el usuario elija, obtener la referencia del objeto correspondiente y llamar a los métodos de reporte de la clase Sistema."*

- **Respuesta de la IA:**

*"A continuación están las dos funciones controladoras solicitadas. Encapsulan la interacción de entrada/salida y delegan la búsqueda de datos al objeto Sistema."*

- **Resultado del Código Generado (main_4.cpp):**

C++

```
// Ejecuta la consulta de ejemplares prestados dada una biblioteca
void ejecutarListarPrestamosPorBiblioteca(Sistema& sistema) {
   std::cout << "\n--- Listar Ejemplares Prestados por Biblioteca ---\n";

   if (sistema.getBibliotecas().empty()) {
      std::cout << "-> No hay bibliotecas registradas en el sistema.\n";
      return;
   }

   sistema.mostrarBibliotecas();
   int indiceBiblioteca = 0;
   std::cout << "Seleccione el numero de la biblioteca (o 0 para salir): ";
   std::cin >> indiceBiblioteca;
```

```
    if (indiceBiblioteca == 0) {
        std::cout << "-> Operacion cancelada. Volviendo al menu principal...\n";
        return;
    }

    if (indiceBiblioteca < 1 || indiceBiblioteca > (int)sistema.getBibliotecas().size()) {
        std::cout << "-> Seleccion invalida de biblioteca. Volviendo al menu principal...\n";
        return;
    }

    Biblioteca* bibliotecaSeleccionada = sistema.getBibliotecas()[indiceBiblioteca - 1];
    sistema.listarPrestamosPorBiblioteca(bibliotecaSeleccionada);
}

// Ejecuta la consulta de libros prestados dado un estudiante
void ejecutarListarPrestamosPorEstudiante(Sistema& sistema) {
    std::cout << "\n--- Listar Libros Prestados por Estudiante ---\n";

    if (sistema.getEstudiantes().empty()) {
        std::cout << "-> No hay estudiantes registrados en el sistema.\n";
        return;
    }

    sistema.mostrarEstudiantes();
    int indiceEstudiante = 0;
    std::cout << "Seleccione el numero del estudiante (o 0 para salir): ";
    std::cin >> indiceEstudiante;

    if (indiceEstudiante == 0) {
        std::cout << "-> Operacion cancelada. Volviendo al menu principal...\n";
        return;
    }

    if (indiceEstudiante < 1 || indiceEstudiante > (int)sistema.getEstudiantes().size()) {
        std::cout << "-> Seleccion invalida de estudiante. Volviendo al menu principal...\n";
        return;
    }

    Estudiante* estudianteSeleccionado = sistema.getEstudiantes()[indiceEstudiante - 1];
    sistema.listarPrestamosPorEstudiante(estudianteSeleccionado);
}
```

**Caso 4: Registro Guiado con Entradas Opcionales y Asociación de Entidades (Autores y Libros)**

- **Prompt enviado a la IA:**

*"¿Cómo puedo implementar dos funciones interactivas en C++ (ejecutarRegistroAutor y ejecutarRegistroLibro) para solicitar datos por consola? Para el autor, si el usuario presiona ENTER sin escribir nada en fecha o nacionalidad, debe asignarse un valor por defecto. Para el libro, debe permitir seleccionar un autor ya registrado mediante una lista numerada o dejarlo como 'Anónimo' si el usuario lo prefiere o si no hay autores disponibles."*

- **Respuesta de la IA:**

"Podemos utilizar std::getline y evaluar si la cadena resulta vacía (fechaNacimiento.empty()) para aplicar los valores por defecto. Para la selección del autor, verificamos si el vector de autores en Sistema contiene elementos, mostramos el menú numerado y capturamos el índice correspondiente."

- **Resultado del Código Generado (main_4.cpp):**

C++

```
// Procesa el registro de un autor
void ejecutarRegistroAutor(Sistema& sistema) {
   std::cout << "\n--- Registrar Autor ---\n";
   std::string nombre, fechaNacimiento, nacionalidad;

   limpiarBuffer();
   std::cout << "Ingrese el nombre del autor (o '0' para salir): ";
   std::getline(std::cin, nombre);

   if (nombre == "0") {
      std::cout << "-> Operacion cancelada.\n";
      return;
   }

   std::cout << "Ingrese la fecha de nacimiento (o presione ENTER para omitir): ";
   std::getline(std::cin, fechaNacimiento);
   if (fechaNacimiento.empty()) fechaNacimiento = "No registrado";

   std::cout << "Ingrese la nacionalidad (o presione ENTER para omitir): ";
   std::getline(std::cin, nacionalidad);
   if (nacionalidad.empty()) nacionalidad = "No registrado";

   // Llama directamente a la funcion unificada
   sistema.registrarAutor(nombre, fechaNacimiento, nacionalidad);
}

// Procesa el registro de un libro
void ejecutarRegistroLibro(Sistema& sistema) {
   std::cout << "\n--- Registrar Libro ---\n";
   Autor* autorSeleccionado = nullptr;

   if (!sistema.getAutores().empty()) {
```

```
    int respAutor = 0;
    std::cout << "Desea asociar un autor existente? (1: Si / 2: No (Anonimo) / 0: Salir): ";
    std::cin >> respAutor;

    if (respAutor == 0) return;
    if (respAutor == 1) {
      sistema.mostrarAutores();
      int indiceAutor = 0;
      std::cout << "Seleccione el numero del autor: ";
      std::cin >> indiceAutor;
      if (indiceAutor >= 1 && indiceAutor <= (int)sistema.getAutores().size()) {
        autorSeleccionado = sistema.getAutores()[indiceAutor - 1];
      }
    }
  }

  std::string titulo;
  int isbn = 0, anio = 0;

  limpiarBuffer();
  std::cout << "Ingrese el titulo del libro: ";
  std::getline(std::cin, titulo);

  std::cout << "Ingrese el ISBN (numero entero): ";
  std::cin >> isbn;

  std::cout << "Ingrese el anio de publicacion (0 si lo desconoce): ";
  std::cin >> anio;

  sistema.registrarLibro(titulo, isbn, anio, autorSeleccionado);
}
```

**Caso 5: Validación de Entradas y Modificación de Stock en la Interfaz (Actualizar Inventario)**

- **Prompt enviado a la IA:**

*"Necesito la función de consola ejecutarActualizacionInventario(Sistema& sistema). Debe permitir al usuario seleccionar una estantería registrada, listar los libros que contiene junto con sus cantidades actuales, dejar seleccionar uno de ellos y solicitar la nueva cantidad. Debe validar que la nueva cantidad sea un número mayor o igual a 0 y cancelar si se selecciona la opción 0."*

- **Respuesta de la IA:**

"Se implementa la navegación por capas: primero se lista y valida el estante, luego se extrae el vector de libros asociados para mostrarlos al usuario con sus cantidades actuales y finalmente se lee y valida nuevaCantidad >= 0 antes de delegar la actualización a sistema.actualizarInventario."

- **Resultado del Código Generado (main_4.cpp):**

C++

```
// Permite actualizar las cantidades de un libro en un estante
```

```
void ejecutarActualizacionInventario(Sistema& sistema) {
    std::cout << "\n--- Actualizar Inventario ---\n";

    if (sistema.getEstantes().empty()) {
        std::cout << "-> No hay estanterias registradas en el sistema.\n";
        return;
    }

    sistema.mostrarEstantes();
    int indiceEstante = 0;
    std::cout << "Seleccione el numero de la estanteria (o 0 para salir): ";
    std::cin >> indiceEstante;

    if (indiceEstante == 0 || indiceEstante < 1 || indiceEstante > (int)sistema.getEstantes().size()) {
        std::cout << "-> Operacion cancelada o seleccion invalida.\n";
        return;
    }

    Estante* estanteSeleccionado = sistema.getEstantes()[indiceEstante - 1];
    const std::vector<Libro*>& librosEstante = estanteSeleccionado->getLibros();
    const std::vector<int>& cantidadesEstante = estanteSeleccionado->getCantidades();

    if (librosEstante.empty()) {
        std::cout << "-> La estanteria seleccionada no tiene libros registrados.\n";
        return;
    }

    std::cout << "\n=== LIBROS EN LA ESTANTERIA: " << estanteSeleccionado->getNombre() << "
===\n";
    for (size_t i = 0; i < librosEstante.size(); ++i) {
        std::cout << i + 1 << ". Libro: " << librosEstante[i]->getTitulo()
                  << " | Cantidad actual: " << cantidadesEstante[i] << "\n";
    }

    int indiceLibro = 0;
    std::cout << "Seleccione el numero del libro a actualizar (o 0 para salir): ";
    std::cin >> indiceLibro;

    if (indiceLibro == 0 || indiceLibro < 1 || indiceLibro > (int)librosEstante.size()) {
        std::cout << "-> Operacion cancelada o seleccion invalida.\n";
        return;
    }

    Libro* libroSeleccionado = librosEstante[indiceLibro - 1];
```

```
    int nuevaCantidad = 0;
    std::cout << "Ingrese la nueva cantidad total disponible (0 o mayor): ";
    std::cin >> nuevaCantidad;

    if (nuevaCantidad < 0) {
        std::cout << "-> Cantidad invalida. Debe ser un numero mayor o igual a 0.\n";
        return;
    }

    sistema.actualizarInventario(estanteSeleccionado, libroSeleccionado, nuevaCantidad);
}
```

**Caso 6: Asignación Interactiva entre Entidades Pertenecientes (Estantería a Biblioteca)**

- **Prompt enviado a la IA:**

"Crea la función ejecutarAsignacionEstanteABiblioteca(Sistema& sistema) para la UI. Debe verificar que existan bibliotecas y estanterías en el sistema, listar ambas mediante índices para que el usuario elija cuál estantería desea asignar a cuál biblioteca y finalmente llamar a sistema.asignarEstanteABiblioteca."

- **Respuesta de la IA:**

"Aquí tienes la función de control de consola. Encapsula las comprobaciones de existencia de bibliotecas y estantes previamente a pedir la interacción del usuario, evitando errores en tiempo de ejecución."

- **Resultado del Código Generado (main_4.cpp):**

C++

```
// Maneja de forma independiente la asignacion de una estanteria a una biblioteca
void ejecutarAsignacionEstanteABiblioteca(Sistema& sistema) {
    std::cout << "\n--- Asignar Estanteria a Biblioteca ---\n";

    if (sistema.getBibliotecas().empty()) {
        std::cout << "-> No hay bibliotecas registradas en el sistema. Registre una biblioteca primero
(Opcion 5).\n";
        return;
    }

    if (sistema.getEstantes().empty()) {
        std::cout << "-> No hay estanterias registradas en el sistema. Registre una estanteria primero
(Opcion 3).\n";
        return;
    }

    sistema.mostrarBibliotecas();
    int indiceBiblioteca = 0;
    std::cout << "Seleccione el numero de la biblioteca (o 0 para salir): ";
    std::cin >> indiceBiblioteca;
```

```
    if (indiceBiblioteca == 0 || indiceBiblioteca < 1 || indiceBiblioteca >
(int)sistema.getBibliotecas().size()) {
        std::cout << "-> Operacion cancelada o seleccion invalida de biblioteca.\n";
        return;
    }

    Biblioteca* bibliotecaSeleccionada = sistema.getBibliotecas()[indiceBiblioteca - 1];

    sistema.mostrarEstantes();
    int indiceEstante = 0;
    std::cout << "Seleccione el numero de la estanteria a asignar (o 0 para salir): ";
    std::cin >> indiceEstante;

    if (indiceEstante == 0 || indiceEstante < 1 || indiceEstante > (int)sistema.getEstantes().size()) {
        std::cout << "-> Operacion cancelada o seleccion invalida de estanteria.\n";
        return;
    }

    Estante* estanteSeleccionado = sistema.getEstantes()[indiceEstante - 1];

    sistema.asignarEstanteABiblioteca(bibliotecaSeleccionada, estanteSeleccionado);
}
```

Aquí tienes los **casos restantes** para completar la documentación de todas las funciones de la interfaz de usuario (UI) presentes en tu archivo main_4.cpp:

**Caso 7: Consultas e Informes de Inventario (ejecutarConsultaLibro, ejecutarListadoInventario, ejecutarConteoInventario)**

- **Prompt enviado a la IA:**

*"Necesito tres funciones auxiliares para el menú de consola que gestionen las consultas del sistema:

1. ejecutarConsultaLibro: Solicitar el título de un libro y buscar su disponibilidad.
2. ejecutarListadoInventario: Desplegar la lista de bibliotecas para que el usuario elija una y se muestre su inventario detallado.
3. ejecutarConteoInventario: Seleccionar una biblioteca y mostrar el conteo total de sus ejemplares.

Todas las funciones deben verificar primero si existen bibliotecas registradas e incluir la opción de cancelar si el usuario ingresa '0'."*

- **Respuesta de la IA:**

"A continuación se presentan las tres funciones de consulta. Encapsulan la interacción de consola, validan la selección dentro de los límites del vector de bibliotecas y delegan la búsqueda de datos al objeto Sistema."

- **Resultado del Código Generado (main_4.cpp):**

C++

```
// Consulta la disponibilidad de un libro por su titulo
void ejecutarConsultaLibro(Sistema& sistema) {
    std::cout << "\n--- Consultar Disponibilidad de Libro ---\n";

    if (sistema.getBibliotecas().empty()) {
        std::cout << "-> No hay bibliotecas registradas en el sistema para consultar.\n";
        return;
    }

    std::string tituloConsultar;
    limpiarBuffer();

    std::cout << "Ingrese el titulo del libro a consultar (o '0' para salir): ";
    std::getline(std::cin, tituloConsultar);

    if (tituloConsultar == "0") {
        std::cout << "-> Operacion cancelada. Volviendo al menu principal...\n";
        return;
    }

    sistema.consultarDisponibilidadLibro(tituloConsultar);
}

// Muestra el listado de inventario de una biblioteca elegida
void ejecutarListadoInventario(Sistema& sistema) {
    std::cout << "\n--- Listado de Inventario ---\n";

    if (sistema.getBibliotecas().empty()) {
        std::cout << "-> No hay bibliotecas registradas en el sistema.\n";
        return;
    }

    sistema.mostrarBibliotecas();
    int indiceBiblio = 0;
    std::cout << "Seleccione el numero de la biblioteca a consultar inventario (o 0 para salir): ";
    std::cin >> indiceBiblio;

    if (indiceBiblio == 0 || indiceBiblio < 1 || indiceBiblio > (int)sistema.getBibliotecas().size()) {
        std::cout << "-> Operacion cancelada o seleccion invalida de biblioteca.\n";
        return;
    }

    sistema.mostrarInventarioBiblioteca(indiceBiblio - 1);
}
```

```
// Realiza el conteo total de ejemplares de una biblioteca
void ejecutarConteoInventario(Sistema& sistema) {
   std::cout << "\n--- Conteo de Inventario ---\n";

   if (sistema.getBibliotecas().empty()) {
      std::cout << "-> No hay bibliotecas registradas en el sistema.\n";
      return;
   }

   sistema.mostrarBibliotecas();
   int indiceBiblioteca = 0;
   std::cout << "Seleccione el numero de la biblioteca para realizar el conteo (o 0 para salir): ";
   std::cin >> indiceBiblioteca;

   if (indiceBiblioteca == 0 || indiceBiblioteca < 1 || indiceBiblioteca >
(int)sistema.getBibliotecas().size()) {
      std::cout << "-> Operacion cancelada o seleccion invalida de biblioteca.\n";
      return;
   }

   sistema.contarInventarioBiblioteca(indiceBiblioteca - 1);
}
```

**Caso 8: Creación e Integración de Ejemplares en Estanterías (ejecutarRegistroEstante, ejecutarRegistroEjemplar)**

- **Prompt enviado a la IA:**

"Escribe dos funciones para el menú interactivo: ejecutarRegistroEstante para capturar el nombre de una nueva estantería, y ejecutarRegistroEjemplar para asociar unidades de un libro existente a una estantería determinada. La segunda función debe verificar que existan tanto libros como estanterías registradas, permitir seleccionar ambos desde listas numeradas e ingresar la cantidad deseada."

- **Respuesta de la IA:**

"Se generan las funciones solicitadas. ejecutarRegistroEjemplar valida la presencia previa de datos en la instancia de Sistema antes de proceder con las solicitudes de entrada por teclado."

- **Resultado del Código Generado (main_4.cpp):**

C++

```
// Procesa el registro de una estanteria
void ejecutarRegistroEstante(Sistema& sistema) {
   std::cout << "\n--- Registrar Estanteria ---\n";
   std::string nombreEstante;

   limpiarBuffer();

   std::cout << "Ingrese el nombre o identificador de la estanteria (o '0' para salir): ";
```

```
    std::getline(std::cin, nombreEstante);

    if (nombreEstante == "0") {
        std::cout << "-> Operacion cancelada. Volviendo al menu principal...\n";
        return;
    }

    sistema.registrarEstante(nombreEstante);
}

// Procesa la asignacion de ejemplares de un libro a un estante
void ejecutarRegistroEjemplar(Sistema& sistema) {
    std::cout << "\n--- Registrar Ejemplar ---\n";

    if (sistema.getLibros().empty()) {
        std::cout << "-> No hay libros registrados en el sistema. Registre un libro primero (Opcion
2).\n";
        return;
    }

    if (sistema.getEstantes().empty()) {
        std::cout << "-> No hay estanterias registradas en el sistema. Registre una estanteria primero
(Opcion 3).\n";
        return;
    }

    sistema.mostrarLibros();
    int indiceLibro = 0;
    std::cout << "Seleccione el numero del libro (o 0 para salir): ";
    std::cin >> indiceLibro;

    if (indiceLibro == 0 || indiceLibro < 1 || indiceLibro > (int)sistema.getLibros().size()) {
        std::cout << "-> Operacion cancelada o seleccion invalida.\n";
        return;
    }

    Libro* libroSeleccionado = sistema.getLibros()[indiceLibro - 1];

    sistema.mostrarEstantes();
    int indiceEstante = 0;
    std::cout << "Seleccione el numero de la estanteria (o 0 para salir): ";
    std::cin >> indiceEstante;

    if (indiceEstante == 0 || indiceEstante < 1 || indiceEstante > (int)sistema.getEstantes().size()) {
```

```
        std::cout << "-> Operacion cancelada o seleccion invalida.\n";
        return;
    }

    Estante* estanteSeleccionado = sistema.getEstantes()[indiceEstante - 1];

    int cantidad = 0;
    std::cout << "Ingrese la cantidad de ejemplares a agregar (o 0 para salir): ";
    std::cin >> cantidad;

    if (cantidad <= 0) {
        std::cout << "-> Operacion cancelada o cantidad invalida. Volviendo al menu principal...\n";
        return;
    }

    sistema.registrarEjemplar(libroSeleccionado, estanteSeleccionado, cantidad);
}
```

**Caso 9: Captura de Entidades Base del Sistema (ejecutarRegistroBiblioteca, ejecutarRegistroEstudiante)**

- **Prompt enviado a la IA:**

"Crea las funciones de consola ejecutarRegistroBiblioteca y ejecutarRegistroEstudiante. Ambas deben utilizar limpiarBuffer() y std::getline para solicitar textos con espacios (como nombres de facultad, biblioteca, código y nombre del estudiante). Si en algún campo el usuario escribe '0', la operación debe cancelarse."

- **Respuesta de la IA:**

"Aquí están las dos funciones de registro primario que leen los datos e invocan los métodos correspondientes de la clase Sistema."

- **Resultado del Código Generado (main_4.cpp):**

C++

```
// Procesa exclusivamente el registro de la biblioteca
void ejecutarRegistroBiblioteca(Sistema& sistema) {
    std::cout << "\n--- Registrar Biblioteca ---\n";
    std::string facultad, nombre;

    limpiarBuffer();

    std::cout << "Ingrese la facultad (o '0' para salir): ";
    std::getline(std::cin, facultad);

    if (facultad == "0") {
        std::cout << "-> Operacion cancelada. Volviendo al menu principal...\n";
        return;
    }
```

```
    std::cout << "Ingrese el nombre de la biblioteca (o '0' para salir): ";
    std::getline(std::cin, nombre);

    if (nombre == "0") {
        std::cout << "-> Operacion cancelada. Volviendo al menu principal...\n";
        return;
    }

    sistema.registrarBiblioteca(facultad, nombre);
}

// Ejecuta la captura interactiva para registrar un estudiante
void ejecutarRegistroEstudiante(Sistema& sistema) {
    std::cout << "\n--- Registrar Estudiante ---\n";
    std::string codigo, nombre;

    limpiarBuffer();
    std::cout << "Ingrese el codigo del estudiante (o '0' para salir): ";
    std::getline(std::cin, codigo);

    if (codigo == "0") {
        std::cout << "-> Operacion cancelada. Volviendo al menu principal...\n";
        return;
    }

    std::cout << "Ingrese el nombre completo del estudiante: ";
    std::getline(std::cin, nombre);

    sistema.registrarEstudiante(codigo, nombre);
}
```

Aquí tienes la continuación estructurada caso por caso del **9 al 14**, correspondiente a las opciones individuales del menú de consola e interacción de UI del archivo main_4.cpp:

**Caso 9: Conteo de Inventario en Biblioteca (Opción 9 del Menú)**

- **Prompt enviado a la IA:**

*"Crea una función de UI en C++ llamada ejecutarConteoInventario que verifique si existen bibliotecas registradas, muestre la lista para selección por teclado y delegue el cálculo del total de ejemplares al método correspondiente del objeto Sistema."*

- **Respuesta de la IA:**

"Aquí tienes la implementación. La función valida la lista de bibliotecas, captura el índice ingresado por el usuario y ejecuta contarInventarioBiblioteca."

- Resultado del Código Generado (main_4.cpp):

C++

```
// Realiza el conteo total de ejemplares de una biblioteca
void ejecutarConteoInventario(Sistema& sistema) {
```

```
    std::cout << "\n--- Conteo de Inventario ---\n";

    if (sistema.getBibliotecas().empty()) {
        std::cout << "-> No hay bibliotecas registradas en el sistema.\n";
        return;
    }

    sistema.mostrarBibliotecas();
    int indiceBiblioteca = 0;
    std::cout << "Seleccione el numero de la biblioteca para realizar el conteo (o 0 para salir): ";
    std::cin >> indiceBiblioteca;

    if (indiceBiblioteca == 0) {
        std::cout << "-> Operacion cancelada. Volviendo al menu principal...\n";
        return;
    }

    if (indiceBiblioteca < 1 || indiceBiblioteca > (int)sistema.getBibliotecas().size()) {
        std::cout << "-> Seleccion invalida de biblioteca. Volviendo al menu principal...\n";
        return;
    }

    sistema.contarInventarioBiblioteca(indiceBiblioteca - 1);
}
```

**Caso 10: Actualización de Inventario por Estantería (Opción 10 del Menú)**

- **Prompt enviado a la IA:**

*"Escribe la función interactiva ejecutarActualizacionInventario en C++. Debe seleccionar una estantería, mostrar sus libros con las cantidades actuales, solicitar el libro a modificar y validar que la nueva cantidad sea un número mayor o igual a 0."*

- **Respuesta de la IA:**

"A continuación se presenta la función. Accede al vector de libros del estante seleccionado, muestra la cantidad disponible actual e invoca el método de actualización del sistema."

- Resultado del Código Generado (main_4.cpp):

C++

```
// Permite actualizar las cantidades de un libro en un estante
void ejecutarActualizacionInventario(Sistema& sistema) {
    std::cout << "\n--- Actualizar Inventario ---\n";

    if (sistema.getEstantes().empty()) {
        std::cout << "-> No hay estanterias registradas en el sistema.\n";
        return;
    }

    sistema.mostrarEstantes();
```

```
    int indiceEstante = 0;
    std::cout << "Seleccione el numero de la estanteria (o 0 para salir): ";
    std::cin >> indiceEstante;

    if (indiceEstante == 0) {
        std::cout << "-> Operacion cancelada. Volviendo al menu principal...\n";
        return;
    }

    if (indiceEstante < 1 || indiceEstante > (int)sistema.getEstantes().size()) {
        std::cout << "-> Seleccion invalida de estanteria.\n";
        return;
    }

    Estante* estanteSeleccionado = sistema.getEstantes()[indiceEstante - 1];
    const std::vector<Libro*>& librosEstante = estanteSeleccionado->getLibros();
    const std::vector<int>& cantidadesEstante = estanteSeleccionado->getCantidades();

    if (librosEstante.empty()) {
        std::cout << "-> La estanteria seleccionada no tiene libros registrados.\n";
        return;
    }

    std::cout << "\n=== LIBROS EN LA ESTANTERIA: " << estanteSeleccionado->getNombre() << "
===\n";
    for (size_t i = 0; i < librosEstante.size(); ++i) {
        std::cout << i + 1 << ". Libro: " << librosEstante[i]->getTitulo()
                  << " | Cantidad actual: " << cantidadesEstante[i] << "\n";
    }

    int indiceLibro = 0;
    std::cout << "Seleccione el numero del libro a actualizar (o 0 para salir): ";
    std::cin >> indiceLibro;

    if (indiceLibro == 0) {
        std::cout << "-> Operacion cancelada. Volviendo al menu principal...\n";
        return;
    }

    if (indiceLibro < 1 || indiceLibro > (int)librosEstante.size()) {
        std::cout << "-> Seleccion invalida de libro.\n";
        return;
    }
```

```
    Libro* libroSeleccionado = librosEstante[indiceLibro - 1];

    int nuevaCantidad = 0;
    std::cout << "Ingrese la nueva cantidad total disponible (0 o mayor): ";
    std::cin >> nuevaCantidad;

    if (nuevaCantidad < 0) {
        std::cout << "-> Cantidad invalida. Debe ser un numero mayor o igual a 0.\n";
        return;
    }

    sistema.actualizarInventario(estanteSeleccionado, libroSeleccionado, nuevaCantidad);
}
```

**Caso 11: Registro de Estudiante (Opción 11 del Menú)**

- **Prompt enviado a la IA:**

*"Necesito la función ejecutarRegistroEstudiante para capturar el código y nombre completo del estudiante mediante std::getline y limpiarBuffer(), ofreciendo la opción de cancelar si ingresa '0' en el código."*

- **Respuesta de la IA:**

"Aquí tienes la función encargada de la captura interactiva para crear un objeto Estudiante en el sistema."

- Resultado del Código Generado (main_4.cpp):

C++

```
// Ejecuta la captura interactiva para registrar un estudiante
void ejecutarRegistroEstudiante(Sistema& sistema) {
    std::cout << "\n--- Registrar Estudiante ---\n";
    std::string codigo, nombre;

    limpiarBuffer();
    std::cout << "Ingrese el codigo del estudiante (o '0' para salir): ";
    std::getline(std::cin, codigo);

    if (codigo == "0") {
        std::cout << "-> Operacion cancelada. Volviendo al menu principal...\n";
        return;
    }

    std::cout << "Ingrese el nombre completo del estudiante: ";
    std::getline(std::cin, nombre);

    sistema.registrarEstudiante(codigo, nombre);
}
```

**Caso 12: Registro Guiado de Préstamo de Libro (Opción 12 del Menú)**

- **Prompt enviado a la IA:**

*"Implementa la función ejecutarRegistroPrestamo para guiar al usuario paso a paso: 1. Elegir estudiante, 2. Elegir biblioteca, 3. Elegir estantería de esa biblioteca, y 4. Elegir libro disponible en esa estantería, permitiendo salir con 0 en cualquier punto."*

- **Respuesta de la IA:**

"Esta función encadena los 4 pasos secuencialmente, obteniendo los punteros de cada objeto y llamando al método registrarPrestamo del sistema."

- Resultado del Código Generado (main_4.cpp):

C++

```
// Ejecuta la captura interactiva para registrar un prestamo
void ejecutarRegistroPrestamo(Sistema& sistema) {
   std::cout << "\n--- Registrar Prestamo de Libro ---\n";

   if (sistema.getEstudiantes().empty()) {
      std::cout << "-> No hay estudiantes registrados. Registre un estudiante primero (Opcion 11).\n";
      return;
   }

   if (sistema.getBibliotecas().empty()) {
      std::cout << "-> No hay bibliotecas registradas en el sistema.\n";
      return;
   }

   // 1. Seleccionar estudiante
   sistema.mostrarEstudiantes();
   int indiceEstudiante = 0;
   std::cout << "Seleccione el numero del estudiante (o 0 para salir): ";
   std::cin >> indiceEstudiante;

   if (indiceEstudiante == 0 || indiceEstudiante < 1 || indiceEstudiante > (int)sistema.getEstudiantes().size()) {
      std::cout << "-> Operacion cancelada o seleccion invalida.\n";
      return;
   }

   Estudiante* estudianteSeleccionado = sistema.getEstudiantes()[indiceEstudiante - 1];

   // 2. Seleccionar biblioteca
   sistema.mostrarBibliotecas();
   int indiceBiblioteca = 0;
   std::cout << "Seleccione el numero de la biblioteca (o 0 para salir): ";
   std::cin >> indiceBiblioteca;
```

```
    if (indiceBiblioteca == 0 || indiceBiblioteca < 1 || indiceBiblioteca >
(int)sistema.getBibliotecas().size()) {
        std::cout << "-> Operacion cancelada o seleccion invalida.\n";
        return;
    }

    Biblioteca* bibliotecaSeleccionada = sistema.getBibliotecas()[indiceBiblioteca - 1];

    const std::vector<Estante*>& estantesBiblio = bibliotecaSeleccionada->getEstantes();
    if (estantesBiblio.empty()) {
        std::cout << "-> La biblioteca seleccionada no tiene estanterias asignadas.\n";
        return;
    }

    // 3. Seleccionar estanteria
    std::cout << "\n=== ESTANTERIAS EN LA BIBLIOTECA: " << bibliotecaSeleccionada->getNombre()
<< " ===\n";
    for (size_t i = 0; i < estantesBiblio.size(); ++i) {
        std::cout << i + 1 << ". ";
        estantesBiblio[i]->print();
    }

    int indiceEstante = 0;
    std::cout << "Seleccione el numero de la estanteria (o 0 para salir): ";
    std::cin >> indiceEstante;

    if (indiceEstante == 0 || indiceEstante < 1 || indiceEstante > (int)estantesBiblio.size()) {
        std::cout << "-> Operacion cancelada o seleccion invalida.\n";
        return;
    }

    Estante* estanteSeleccionado = estantesBiblio[indiceEstante - 1];

    // 4. Seleccionar libro
    const std::vector<Libro*>& librosEstante = estanteSeleccionado->getLibros();
    const std::vector<int>& cantidadesEstante = estanteSeleccionado->getCantidades();

    if (librosEstante.empty()) {
        std::cout << "-> La estanteria seleccionada no tiene libros registrados.\n";
        return;
    }

    std::cout << "\n=== LIBROS DISPONIBLES EN LA ESTANTERIA: " << estanteSeleccionado-
>getNombre() << " ===\n";
```

```
  for (size_t i = 0; i < librosEstante.size(); ++i) {
    std::cout << i + 1 << ". Libro: " << librosEstante[i]->getTitulo()
          << " | Cantidad disponible: " << cantidadesEstante[i] << "\n";
  }

  int indiceLibro = 0;
  std::cout << "Seleccione el numero del libro a prestar (o 0 para salir): ";
  std::cin >> indiceLibro;

  if (indiceLibro == 0 || indiceLibro < 1 || indiceLibro > (int)librosEstante.size()) {
    std::cout << "-> Operacion cancelada o seleccion invalida.\n";
    return;
  }

  Libro* libroSeleccionado = librosEstante[indiceLibro - 1];

  sistema.registrarPrestamo(estudianteSeleccionado, bibliotecaSeleccionada,
estanteSeleccionado, libroSeleccionado);
}
```

**Caso 13: Reporte de Ejemplares Prestados por Biblioteca (Opción 13 del Menú)**

- **Prompt enviado a la IA:**

*"Genera la función ejecutarListarPrestamosPorBiblioteca para la interfaz. Debe validar la existencia de bibliotecas, pedir al usuario elegir una mediante su número en lista y llamar al reporte correspondiente de la clase Sistema."*

- **Respuesta de la IA:**

"Aquí está la función controladora de UI. Filtra las bibliotecas registradas y ejecuta la consulta de préstamos activos."

- Resultado del Código Generado (main_4.cpp):

C++

```
// Ejecuta la consulta de ejemplares prestados dada una biblioteca
void ejecutarListarPrestamosPorBiblioteca(Sistema& sistema) {
  std::cout << "\n--- Listar Ejemplares Prestados por Biblioteca ---\n";

  if (sistema.getBibliotecas().empty()) {
    std::cout << "-> No hay bibliotecas registradas en el sistema.\n";
    return;
  }

  sistema.mostrarBibliotecas();
  int indiceBiblioteca = 0;
  std::cout << "Seleccione el numero de la biblioteca (o 0 para salir): ";
  std::cin >> indiceBiblioteca;

  if (indiceBiblioteca == 0) {
```

```
    std::cout << "-> Operacion cancelada. Volviendo al menu principal...\n";
    return;
  }

  if (indiceBiblioteca < 1 || indiceBiblioteca > (int)sistema.getBibliotecas().size()) {
    std::cout << "-> Seleccion invalida de biblioteca. Volviendo al menu principal...\n";
    return;
  }

  Biblioteca* bibliotecaSeleccionada = sistema.getBibliotecas()[indiceBiblioteca - 1];
  sistema.listarPrestamosPorBiblioteca(bibliotecaSeleccionada);
}
```

**Caso 14: Reporte de Libros Prestados por Estudiante (Opción 14 del Menú)**

- **Prompt enviado a la IA:**

*"Crea la función ejecutarListarPrestamosPorEstudiante. Debe mostrar la lista de estudiantes registrados, solicitar seleccionar uno mediante índice y mostrar todos los libros que dicho estudiante tiene prestados actualmente."*

- **Respuesta de la IA:**

"A continuación tienes la función que gestiona la búsqueda de préstamos vinculados a un estudiante determinado."

- Resultado del Código Generado (main_4.cpp):

C++

```
// Ejecuta la consulta de libros prestados dado un estudiante
void ejecutarListarPrestamosPorEstudiante(Sistema& sistema) {
  std::cout << "\n--- Listar Libros Prestados por Estudiante ---\n";

  if (sistema.getEstudiantes().empty()) {
    std::cout << "-> No hay estudiantes registrados en el sistema.\n";
    return;
  }

  sistema.mostrarEstudiantes();
  int indiceEstudiante = 0;
  std::cout << "Seleccione el numero del estudiante (o 0 para salir): ";
  std::cin >> indiceEstudiante;

  if (indiceEstudiante == 0) {
    std::cout << "-> Operacion cancelada. Volviendo al menu principal...\n";
    return;
  }

  if (indiceEstudiante < 1 || indiceEstudiante > (int)sistema.getEstudiantes().size()) {
    std::cout << "-> Seleccion invalida de estudiante. Volviendo al menu principal...\n";
    return;
```

```
    }

    Estudiante* estudianteSeleccionado = sistema.getEstudiantes()[indiceEstudiante - 1];
    sistema.listarPrestamosPorEstudiante(estudianteSeleccionado);
}
```